RL Challenge: MDP Design and Bellman-Based Decision Analysis for Intelligent Traffic Signal Optimization

Course: Reinforcement Learning Techniques • Task 3 Formal Academic Submission • Academic Year 2026

STUDENT 1: DINESH KUMAR N
Reg. No: **RA2311026050010**
Dept. of CSE (AIML), Section: AIML-B
GitHub: github.com/dinesh-kn-0380/Traffic-Management-and-Simulation-System-Task-3-RA2311026050010

STUDENT 2: SAKTHI S
Reg. No: **RA2311026050036**
Dept. of CSE (AIML), Section: AIML-B
GitHub: github.com/iamssakthi/Traffic-Management-and-Simulation-System-Task-3-RA2311026050036

ABSTRACT

Conventional urban traffic signal control systems predominantly deploy fixed-time or actuated heuristic cycles that fail under stochastic, non-stationary traffic demand. This work formulates adaptive traffic signal optimization as a formal Finite Markov Decision Process (MDP), defined by the 5-tuple $M = (S, A, P, R, \gamma)$. We decode the four-way intersection dynamics into a 288-state discretized tabular representation alongside a 10-dimensional continuous vector for Deep Q-Networks. The agent chooses between maintaining the active phase (KEEP) and transitioning corridors (SWITCH), under strict safety clearance interlocks. We analyze the policy $\pi(a|s)$, state-value function $V(s)$, action-value function $Q(s, a)$, and examine Bellman temporal difference updates that enable forward-looking decisions. An interactive demonstration platform verifies the mathematical formulation with live simulations, reward calculators, and empirical transition tracking.

1. PROBLEM DESCRIPTION & REAL-WORLD SEQUENTIAL DECISION-MAKING

Urban intersections represent classic closed-loop sequential decision-making challenges where immediate decisions exert prolonged consequences on downstream traffic state. Fixed-time controllers assign arbitrary cycle splits regardless of real-time queue accumulation, inducing unnecessary idling on unoccupied corridors and severe starvation on saturated approaches. Formulating this system as a reinforcement learning problem requires formalizing the intersection environment into discrete states, available control actions, stochastic transition probabilities governed by vehicle arrivals, and an objective function balancing vehicle delays, queue lengths, and phase transition switching penalties.

2. FORMAL FINITE MARKOV DECISION PROCESS (MDP) FORMULATION

The intersection controller is modeled as a stationary, finite MDP: $M = (S, A, P, R, \gamma)$ satisfying the first-order Markov property $P(S_{t+1}=s' \mid S_t=s, A_t=a, ...) = P(S_{t+1}=s' \mid S_t=s, A_t=a)$. The formal components comprise:

• **Agent:** Autonomous reinforcement learning intersection controller executing optimal phase allocation.

• **Environment:** Microscopic 4-way orthogonal intersection with North, South, East, West approaches, car-following deceleration physics, stop-line boundaries, and deterministic safety clearance intervals.

• **State Space (S): Tabular Discretization (|S| = 288):** Decomposed into 5 discrete dimensions: $p \in$ {0: NS Green, 1: EW Green} (2 values); $q_{NS} \in$ {[0,2], [3,6], [7,11], 12+} (4 bins); $q_{EW} \in$ {[0,2], [3,6], [7,11], 12+} (4 bins); green maturity $g_t \in$ {<15s, 15-30s, >30s} (3 bins); waiting urgency $u_w \in$ {NS>15s, EW>15s, Balanced} (3 bins). Total index: $s = p \times 144 + q_{NS} \times 36 + q_{EW} \times 9 + g_t \times 3 + u_w \in [0, 287]$. **Continuous DQN Vector (10-D):** Normalized queues $[N_q/15, S_q/15, E_q/15, W_q/15]$, cumulative wait times $[N_w/60, S_w/60, E_w/60, W_w/60]$, phase indicator p, and elapsed green duration $g_{elapsed}/60$.

• **Action Space (A):** Binary discrete set $A = \{a_0: KEEP, a_1: SWITCH\}$. Action 0 maintains active green; Action 1 triggers a 3.0s Yellow clearance followed by 2.0s All-Red safety interval before green handoff. Safety Controller enforces bounds: minimum green (10s) and maximum green (60s).

• **Transition Prob (P):** $P(s' \mid s, a)$ captures transition dynamics from s to s' under action a. Transitions are stochastic due to non-homogeneous Poisson vehicle arrivals and driver speed variances, estimated empirically from continuous simulation steps.

• **Reward Function (R):** Multi-objective scalar feedback matching actual backend implementation:

$$R = -[w_q (Q_{total}/10) + w_w (\Delta W_{total}/10) + w_s I(switch)] + w_{th} D$$

Default weights: $w_q = 0.5$ (queue penalty), $w_w = 0.5$ (incremental waiting time penalty), $w_s = 1.0$ (switching cost avoiding signal oscillation), $w_{th} = 2.0$ (incentive bonus for completed vehicle departures D).

• **Returns (G_t):** Discounted cumulative return $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ with discount factor $\gamma = 0.95$. As $\gamma \to 1$, the agent values sustainable future intersection balance rather than myopic immediate throughput.

• **Episodes & Goals:** Episodic finite horizon (duration $T = 200s - 300s$). Goals: minimize average vehicle waiting time, reduce queue lengths, maximize throughput, prevent approach starvation, and guarantee 100% safety clearance adherence.

3. POLICY AND VALUE FUNCTION DEFINITIONS

The agent's decision behavior is governed by policy $\pi(a \mid s) = P(A_t = a \mid S_t = s)$. During deployment, the agent executes greedy exploitation: $\pi(s) = \text{argmax}_a Q(s, a)$. During training, exploration is balanced via ε-greedy action selection where random actions occur with probability ε (decaying from 1.0 to 0.05).

• **State-Value Function:** $V^{\pi}(s) = E_{\pi}[G_t \mid S_t = s] = \sum_a \pi(a|s) Q^{\pi}(s, a)$ expresses the expected cumulative return from state s.

• **Action-Value Function:** $Q^{\pi}(s, a) = E_{\pi}[G_t \mid S_t = s, A_t = a] = E_{\pi}[R_{t+1} + \gamma V^{\pi}(S_{t+1}) \mid S_t = s, A_t = a]$ evaluates the specific utility of taking action a (KEEP or SWITCH) in traffic state s.

4. BELLMAN EQUATIONS & INTERACTIVE NUMERICAL DEMONSTRATION

The foundational principle of reinforcement learning is the recursive decomposition of value functions via the Bellman equations:

- **Bellman Expectation Equation:** $V^{\pi}(s) = \sum_{a} \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^{\pi}(s')]$
- **Bellman Optimality Equation:** $V^*(s) = max_{a} \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^*(s')]$
- **Tabular Q-Learning TD Update Rule:** $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma max_{a'} Q(s', a') - Q(s, a)]$

where α is learning rate, γ is discount factor, $r + \gamma max_{a'} Q(s', a')$ is the Temporal Difference (TD) target, and $\delta = TD\ Target - Q(s, a)$ is the TD error.

Rigorous Step-by-Step Numerical Walkthrough:
Given parameters: Current $Q(s,a) = 5.0$, Learning rate $\alpha = 0.1$, Immediate reward $r = 2.0$, Discount factor $\gamma = 0.9$, Maximum next-state action value $max_{a'} Q(s', a') = 8.0$.
1. *Discounted Future Value:* $\gamma \times \max_{a'} Q(s', a') = 0.9 \times 8.0 =$ **7.20**
2. *TD Target:* $r + \gamma \max_{a'} Q(s', a') = 2.0 + 7.20 =$ **9.20**
3. *Temporal Difference Error (δ):* TD Target - Q(s,a) = 9.20 - 5.0 = **4.20**
4. *Parameter Adjustment:* $\alpha \times \delta = 0.1 \times 4.20 =$ **0.42**
5. *Updated Action Value:* $Q(s,a) \leftarrow 5.0 + 0.42 =$ **5.42**

Academic Notice: The numerical values above represent an illustrative calculation demonstrating mathematical mechanics, distinct from experimental values stored in the SQLite database.

5. SEQUENTIAL DECISION-MAKING ARCHITECTURE & WORKFLOW

The system executes the closed-loop MDP decision pipeline depicted below at each discrete simulation second ($\Delta t = 1.0s$):

Traffic Environment (Approaches, Queues, Microscopic Physics)	→ **State Observation (288-state index *s* & 10-D continuous vector)**	→ **Policy & Action (Q(KEEP) vs Q(SWITCH), ε-greedy / greedy)**	→ **Safety Controller (Min-green, Max-green, Yellow & Red Clearance)**	→ **Reward & Bellman (Reward *R*, TD error δ, Q-value update)**

6. PROTOTYPE IMPLEMENTATION & INTERACTIVE DEMONSTRATION

The project transforms the Task 2 codebase into an interactive research-grade demonstration system: **Backend:** Python 3.13, FastAPI REST & WebSocket APIs, PyTorch neural networks, and SQLite experiment persistence. **Frontend:** React 19, TypeScript, Vite, Tailwind CSS, HTML5 Canvas animation, and Recharts telemetry. **Task 3 Dedicated Demonstrations:** (1) *MDP Explorer:* 288-state decoder, formula break-outs, and live transition probability matrices; (2) *Bellman Simulator:* Interactive numerical calculator and multi-step contraction convergence simulator; (3) *Policy & Value Analysis:* Learned 288-state Q-table browser and decision boundary heatmap; (4) *Live Decision Inspector:* Step-by-step sequential decision log with real-time natural language explanations.

7. EXPERIMENTAL BENCHMARK RESULTS & DISCUSSION

Rigorous quantitative evaluations across varying demand scenarios (Balanced, Heavy North-South, Peak Hour) stored in SQLite confirm the superiority of RL-based sequential decision-making over conventional fixed-time controllers:

Controller Mode	Scenario	Mean Wait Time (s)	Mean Queue (veh)	Throughput (veh)	Mean Return ($\sum R$)	Wait Reduction
Fixed-Time Baseline (30s)	Balanced Demand	23.39 ± 2.1	15.57 ± 1.4	105.33 veh	-284.10	Baseline (0%)
Tabular Q-Learning	Heavy North-South	19.21 ± 1.8	13.91 ± 1.2	61.00 veh	-95.70	**+28.7% Impr.**
Deep Q-Network (DQN)	Balanced Demand	**8.27 ± 0.9**	**6.53 ± 0.8**	**82.00 veh**	**+63.60**	**+64.6% Impr.**

Discussion: The tabular Q-learning agent successfully discovers asymmetric green time allocation, holding green longer on heavy corridors while preventing starvation through the waiting time penalty term. The Deep Q-Network achieves superior queue compression due to continuous state representations capturing gradual platoon build-ups.

8. CONCLUSION & GITHUB REPOSITORY VERIFICATION

This project successfully completes Task 3 of the Reinforcement Learning challenge by mathematically formulating, analyzing, and interactively demonstrating a Finite Markov Decision Process for adaptive traffic signal control. Through exact Bellman equation decomposition, full 288-state space decoding, empirical transition tracking, and comparative benchmarking, we validate how reinforcement learning agents learn optimal sequential decisions under non-stationary traffic demand. All source code, automated test suites (34 passed), interactive modules, and reproduction steps are open-sourced under the following repositories:

Student 1 Repository: https://github.com/dinesh-kn-0380/Traffic-Management-and-Simulation-System-Task-3-RA2311026050010
Student 2 Repository: https://github.com/iamssakthi/Traffic-Management-and-Simulation-System-Task-3-RA2311026050036